



Univerzitet u Sarajevu
Elektrotehnički fakultet
Sarajevo

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET

P16: BVH RAY TRACER

Projektni izvještaj

CHECKPOINT 2

Predmet: Paralelni računarski sistemi
Mentor: Tarik Hrnjić, MA

Autori

Dino Burić - 2502/2026

Ali Fazlić - 2590/2026

Zijad Doglod - 2576/2026

Ajdin Dželo - 2527/19540

Sarajevo, novembar 2026.

Sažetak

U drugoj fazi projekta P16: BVH Ray Tracer pripremili smo polaznu sekvencijalnu implementaciju praćenja zraka (ray tracing) s BVH strukturom. Kao osnovu smo uzeli kod iz knjige Ray Tracing: The Next Week i javnog repozitorija serijala Ray Tracing in One Weekend, a zatim ga pročistili tako da u projektu ostane samo ono što je potrebno za temu BVH ubrzanja. Iz originalnog repozitorija izbacili smo druge dvije knjige, web dokumentaciju, pomoćne slike i programe, a iz samog koda uklonili smo zamućenje pokreta (motion blur). Dodali smo jedinstven CMake build, mjerenje vremena renderovanja, opisne komentare i uputstvo na bosanskom jeziku. Prvo mjerenje pokazuje da BVH skraćuje vrijeme renderovanja približno 2,8 puta na sceni s oko 480 sfera i približno 38 puta na finalnoj sceni s oko 3400 primitiva. Program smo zatim proširili prekidačem kojim se pri pokretanju biraju scena i upotreba BVH-a, bez ponovnog kompajliranja; slika se snima kao PNG i automatski otvara, a vrijeme izgradnje scene i renderovanja ispisuje se odvojeno. Ovaj dokument opisuje šta je urađeno, šta je izbačeno i zašto, te predstavlja osnovu za paralelizaciju u narednoj fazi.

1. Uvod

U prvoj fazi (Checkpoint 1) pregledali smo literaturu o BVH strukturama i planirali izradu ray tracera u C++-u, na kojem ćemo porediti sekvencijalnu izvedbu bez BVH-a, sekvencijalnu izvedbu s BVH-om i paralelnu izvedbu. Cilj druge faze bio je dobiti ispravan, pregledan i mjerljiv polazni program na kojem se ta poređenja mogu izvoditi.

Umjesto pisanja renderera od nule, odlučili smo poći od koda iz knjige *Ray Tracing: The Next Week* [1], koji je javno dostupan u repozitoriju serijala *Ray Tracing in One Weekend* [2]. Ta knjiga uvodi upravo AABB kutije i BVH stablo, pa je njen kod prirodna osnova za naš projekat. Originalni repozitorij je, međutim, napravljen za objavljivanje cijelog serijala knjiga i sadrži mnogo sadržaja koji nam nije potreban. Zato smo kod prenijeli u naš repozitorij i pročistili ga. U nastavku opisujemo strukturu projekta, sve što smo izbacili i razloge za to, šta smo dodali te rezultate prvog mjerenja.

2. Struktura projekta

Originalni repozitorij [2] sadrži izvorni kod za tri knjige (`InOneWeekend`, `TheNextWeek` i `TheRestOfYourLife`), tekst knjiga u Markdeep formatu, stilove i skripte za web stranicu, 107 slika i niz pratećih dokumenata. Naš repozitorij zadržava samo jedan program i organizovan je oko foldera `Project/` za kod i foldera `docs/` za izvještaje po fazama. Poređenje je dato u tabeli 1.

Tabela 1. Poređenje strukture originalnog i našeg repozitorija.

Originalni repozitorij	**Naš repozitorij**
`src/InOneWeekend/`, `src/TheNextWeek/`, `src/TheRestOfYourLife/`	`Project/src/` (samo kod iz `TheNextWeek`)
`src/external/` (stb biblioteke [6])	`Project/src/external/` (nepromijenjeno)
`images/` (107 slika: ilustracije za knjige i teksture)	`Project/images/` (samo `earthmap.jpg`)
`books/`, `style/`, `index.html`, `favicon.png`	izbačeno

Originalni repozitorij	Naš repozitorij
`CMakeLists.txt` s 13 izvršnih ciljeva	`Project/CMakeLists.txt` s jednim ciljem `raytracer`
`README.md`, `CHANGELOG.md`, `CONTRIBUTING.md`, `PRINTING.md`, `COPYING.txt`	`README.md` (na bosanskom), `LICENSE`, `.gitignore`
nema	`docs/checkpoint 1/`, `docs/checkpoint 2/` (izvještaji)

Folder `Project/src/` sadrži `main.cc` s devet ugrađenih scena i 18 zaglavlja (`.h` datoteka). Za BVH su najvažnija zaglavlja `aabb.h` i `bvh.h`, a ostala pokrivaju geometriju (`sphere.h`, `quad.h`, `constant_medium.h`), materijale (`material.h`), teksture (`texture.h`, `perlin.h`, `rtw_stb_image.h`), snimanje slike (`rtw_stb_image_write.h`), kameru (`camera.h`) i matematičke pomoćne klase (`vec3.h`, `ray.h`, `interval.h`, `rtweekend.h`). Renderovane slike se pišu u `Project/output/`, koji je, kao i build folder, isključen iz Gita pomoću `.gitignore`.

3. Šta smo izbacili i zašto

3.1. Druge dvije knjige serijala

Iz originalnog repozitorija preuzeli smo samo kod iz foldera `src/TheNextWeek/`. Kod iz `InOneWeekend` je u potpunosti obuhvaćen kodom iz `TheNextWeek`, jer druga knjiga nastavlja na prvu i proširuje iste datoteke; zadržavanje obje verzije značilo bi dvije gotovo identične kopije istih klasa. Kod iz `TheRestOfYourLife` bavi se Monte Carlo integracijom i uzorkovanjem po važnosti (importance sampling): uvodi klase `pdf` i `onb` te osam samostalnih demonstracionih programa (`pi.cc`, `integrate_x_sq.cc`, `sphere_plot.cc` i drugi). Te tehnike poboljšavaju kvalitet slike pri istom broju uzoraka, ali ne utiču na BVH, koji je tema projekta. Njihovo uključivanje bi otežalo tumačenje mjerenja, jer bi se promjene u vremenu mogle pripisati drugačijem načinu uzorkovanja umjesto akceleracijskoj strukturi.

3.2. Web dokumentacija i prateći sadržaj

Folderi `books/` i `style/`, te datoteke `index.html` i `favicon.png` služe za generisanje web stranice s tekstom knjiga. Folder `images/` sadrži 106 ilustracija za knjige (`fig-*.jpg`, naslovnice) i samo jednu teksturu koju program zaista koristi, `earthmap.jpg`. Datoteke `CHANGELOG.md`, `CONTRIBUTING.md` i `PRINTING.md` opisuju razvoj i štampanje knjiga. Ništa od toga ne učestvuje u izgradnji programa, pa smo zadržali samo `earthmap.jpg`, koju koriste scene 3 (`earth`) i 9 (`final_scene`). Knjige ostaju dostupne na izvornoj adresi i navedene su u literaturi [1, 2].

3.3. Zamućenje pokreta (motion blur)

Jedina funkcionalna izmjena koda je uklanjanje zamućenja pokreta. U originalu svaka zraka nosi vrijeme `tm`, kamera za svaki uzorak bira nasumično vrijeme unutar intervala okidača, a pokretna sfera svoj centar čuva kao zraku i računa trenutni položaj kao `center.at(r.time())`. Izmjene su sljedeće:

- `ray.h`: uklonjeni su član `tm`, metoda `time()` i konstruktor s tri parametra; zraka sada ima samo početnu tačku i smjer.
- `camera.h`: `get_ray()` više ne poziva `random_double()` za vrijeme zrake.
- `sphere.h`: uklonjen je konstruktor pokretne sfere; centar je obična tačka (`point3`) umjesto zrake, pa se pri svakom testu presjeka ne računa njegov položaj.

- `material.h` i `hittable.h`: rasijane zrake u četiri materijala koji rasijavaju svjetlost, kao i zrake u transformacijama `translate` i `rotate_y`, više ne prenose vrijeme.
- `main.cc`: pokretne sfere u scenama `bouncing_spheres` i `final_scene` zamijenjene su statičnim sferama u početnom položaju. Originalne linije su zadržane kao komentar radi poređenja.

Razloga je nekoliko. Prvo, zamućenje pokreta nije dio teme projekta. Drugo, utiče na kvalitet samog BVH-a: AABB pokretne sfere mora obuhvatiti sferu u oba krajnja položaja, pa je kutija veća od stvarnog objekta i zraka je češće pogađa bez stvarnog presjeka. Time bi mjerenja BVH-a bila opterećena efektom koji ne potiče od same strukture. Treće, manje podataka po zraci i jedan poziv generatora slučajnih brojeva manje po uzorku pojednostavljuju kod koji ćemo u narednoj fazi paralelizovati. Posljedica je da scena 1 (`bouncing_spheres`) više ne pokazuje efekat zbog kojeg je napravljena u knjizi, ali je zadržana kao testna scena jer sadrži oko 480 sfera, što je pogodno za mjerenje BVH-a.

3.4. Zaglavlja s licencom i suvišni komentari

Na početku skoro svake datoteke originalni kod ima isto zaglavlje s izjavom o CC0 licenci (javno dobro). Ta zaglavlja smo uklonili i na njihovo mjesto, gdje je to imalo smisla, upisali kratak opis uloge datoteke. Izvor koda je naveden u `README.md` i u ovom izvještaju [1, 2]. Uklonjen je i komentar o „C++ suptilnosti“ u konstruktoru `bvh_node`, dok je objašnjenje kopiranja liste zadržano.

4. Šta smo dodali

4.1. Jedinstven build

Originalni `CMakeLists.txt` gradi 13 izvršnih ciljeva (tri renderera i demonstracione programe iz treće knjige). Naš gradi samo jedan, `raytracer`, iz `src/main.cc`. Na Linuxu i macOS-u, ako tip builda nije naveden, automatski se bira Release, jer je Debug build višestruko sporiji i mjerenja na njemu ne bi imala smisla. Standard ostaje C++11, kao u originalu.

4.2. Mjerenje vremena renderovanja

U `main()` je dodano mjerenje vremena pomoću `std::chrono::steady_clock`. U prvoj verziji mjerilo se samo ukupno vrijeme izvršavanja odabrane scene. U kasnijoj verziji, opisanoj u sekciji 6, izgradnja scene (uključujući BVH stablo) i renderovanje mjere se odvojeno.

4.3. Komentari i dokumentacija

Svako zaglavlje sada počinje opisom onoga što definiše, a klase materijala i tekstura te pomoćne funkcije u `vec3.h` (`reflect`, `refract`, `random_unit_vector` i druge) imaju kratke komentare o namjeni. Komentari u kodu su na engleskom, u skladu s nazivima u kodu. `README.md` je napisan na bosanskom i opisuje mogućnosti programa, strukturu foldera, build na Windowsu i Linuxu/macOS-u, pokretanje, odabir scene i parametre finalne scene.

5. BVH u našem kodu

BVH je implementiran u dvije datoteke. U `aabb.h` se nalazi klasa `aabb`, kutija zadana s tri intervala, po jedan za svaku osu. Presjek sa zrakom računa se metodom ploča (slab method):

za svaku osu se izračuna interval parametra `t` u kojem je zraka između dvije paralelne ravni, a zraka pogađa kutiju ako se sva tri intervala preklapaju. Vrlo tanke kutije (na primjer oko četverougla) se malo prošire kako bi se izbjegli problemi s numeričkom preciznošću.

U `bvh.h` se nalazi klasa `bvh_node`. Izgradnja je rekurzivna: za skup objekata računa se zajednička kutija, odabere se njena najduža osa, objekti se sortiraju po donjoj granici svojih kutija na toj osi i dijele na dvije polovine po broju objekata. Čvor s jednim ili dva objekta postaje list. Pri obilasku se prvo testira kutija čvora; ako je zraka promaši, cijelo podstablo se preskače. Ako lijevo dijete ima presjek, desno se testira samo do udaljenosti tog presjeka, čime se odbacuju dalji objekti.

Ovakva podjela je jednostavna i brzo se gradi, ali ne uzima u obzir površinu kutija, kao što to rade heuristike zasnovane na površini (SAH) opisane u literaturi [3, 5]. Zato je ona dobra polazna tačka za poređenje s boljim načinima izgradnje.

BVH koriste samo dvije funkcije scena: `bouncing_spheres`, u kojoj se cijela scena pretvara u jedno stablo, i `final_scene`, u kojoj se grade dva stabla, jedno za 400 kutija poda i jedno za oblak od 1000 malih sfera. Ostalih sedam scena ima najviše osam objekata, pa su svi u običnoj listi `hittable_list`. Kod tako malih scena BVH ne donosi ubrzanje: lista od nekoliko objekata prođe se brzo, a testiranje kutija u stablu ima svoju cijenu. Ovakva podjela preuzeta je iz originalnog koda [1].

6. Prekidač za BVH i pokretanje programa

Za mjerenja u sekciji 7 verziju bez BVH-a dobijali smo ručnom izmjenom koda i ponovnim kompajliranjem. Takav postupak je spor i podložan greškama, na primjer ako se izmjena zaboravi vratiti. Zato smo program proširili tako da se scena i upotreba BVH-a biraju pri pokretanju. Izmjene su razvijene u zasebnoj Git grani `bvhswitch`, a standard jezika je i dalje C++11.

6.1. Odabir scene i BVH-a pri pokretanju

Pokrenut bez argumenata, program ispisuje listu scena i pita koju da renderuje. Ako odabrana scena koristi BVH, pita i `Koristiti BVH? (d/n)`, pri čemu prazan unos znači „da“. Neispravan unos se odbija i pitanje se ponavlja. Scene u meniju prikazane su u tabeli 2. Finalna scena se pojavljuje dva puta, kao brza testna verzija i kao verzija visokog kvaliteta, pa BVH koriste scene 0, 1 i 9.

Tabela 2. Scene u meniju programa (uzorci po pikselu / dubina) i upotreba BVH-a.

#	Scena	Sadržaj	BVH
0	`final_scene`, brza (400 px, 250 / 4)	oko 3400 primitiva	da
1	`bouncing_spheres`	oko 480 sfera	da
2	`checkered_spheres`	2 sfere	ne
3	`earth`	1 sfera	ne
4	`perlin_spheres`	2 sfere	ne
5	`quads`	5 četverouglova	ne
6	`simple_light`	4 objekta	ne
7	`cornell_box`	6 zidova i 2 kutije	ne
8	`cornell_smoke`	6 zidova i 2 kutije od dima	ne
9	`final_scene`, visok kvalitet (800 px, 10000 / 40)	oko 3400 primitiva	da

Isti izbor može se zadati argumentima komandne linije, i tada program ne postavlja pitanja. Na primjer, `raytracer 1 --no-bvh --no-open` renderuje scenu 1 bez BVH-a i ne otvara sliku, što je pogodno za skripte koje automatski izvode niz mjerenja.

6.2. Prekidač u kodu

U `bvh.h` je dodana globalna zastavica `use_bvh` i funkcija `accelerate()`, koja za zadanu listu objekata vraća BVH stablo (`bvh_node`) ako je zastavica uključena, a inače običnu listu (`hittable_list`). Obje klase su podklase klase `hittable`, pa ostatak programa ne mora znati koju je varijantu dobio. U `main.cc` su tri mjesta na kojima se ranije direktno gradio `bvh_node` zamijenjena pozivom `accelerate()`: cijela scena u `bouncing_spheres` te kutije poda i oblak sfera u `final_scene`. Zastavica je deklarisana kao `extern bool` u zaglavlju i definisana u `main.cc`, jer jednostavniji oblik (`inline` varijabla) zahtijeva C++17.

6.3. Izlaz u PNG formatu i automatsko otvaranje

Ranije je program sliku ispisivao u tekstualnom PPM formatu na standardni izlaz, pa ju je trebalo preusmjeriti u datoteku znakom >, koji se u PowerShellu ponaša drugačije nego u cmd-u. Uz to, PPM se na Windowsu ne može otvoriti bez dodatnog programa. Sada `camera::render()` boje piksela upisuje u bafer, a na kraju ga snima kao PNG pomoću biblioteke `stb_image_write` [6]. Ta biblioteka je već postojala u preuzetom kodu, ali se nije koristila. Funkcija `write_color` u `color.h` zamijenjena je funkcijom `color_to_bytes`, koja primjenjuje istu gama korekciju i ograničavanje na [0, 255], ali rezultat upisuje u bafer.

Slika se snima u folder `output/`, koji program sam pravi ako ne postoji. Ime datoteke sadrži broj scene i oznaku `_bvh` ili `_bez_bvh`, na primjer `1_bouncing_spheres_bez_bvh.png`, pa se rezultati dva pokretanja ne prepisuju. Nakon snimanja slika se otvara u podrazumijevanom pregledniku slika operativnog sistema (naredbe `start` na Windowsu, `open` na macOS-u i `xdg-open` na Linuxu).

6.4. Odvojeno mjerenje izgradnje i renderovanja

Funkcije scena više same ne pozivaju renderovanje, nego samo popunjavaju listu objekata i podešavaju kameru. Zahvaljujući tome `main()` mjeri dvije faze odvojeno: izgradnju scene, koja uključuje i izgradnju BVH stabla, te renderovanje sa snimanjem slike. Na kraju se ispisuje sažetak:

```
Scena:            1 - bouncing_spheres
BVH:              da
Izgradnja scene:  0.000 s
Renderovanje:     4.676 s
Ukupno:           4.676 s
Slika:            output/1_bouncing_spheres_bvh.png
```

Izgradnja BVH-a za scene ovih veličina traje oko 10 ms ili manje, pa je zanemariva u odnosu na renderovanje. To se može promijeniti kod scena s mnogo više objekata, što ćemo pratiti u narednoj fazi.

7. Prvo mjerenje

Da bismo provjerili da BVH zaista ubrzava renderovanje, uporedili smo scene `bouncing_spheres` i `final_scene` s BVH-om i bez njega. Ovo mjerenje izvedeno je prije uvođenja prekidača, ručnom izmjenom koda: u `bouncing_spheres()` je zakomentarisana

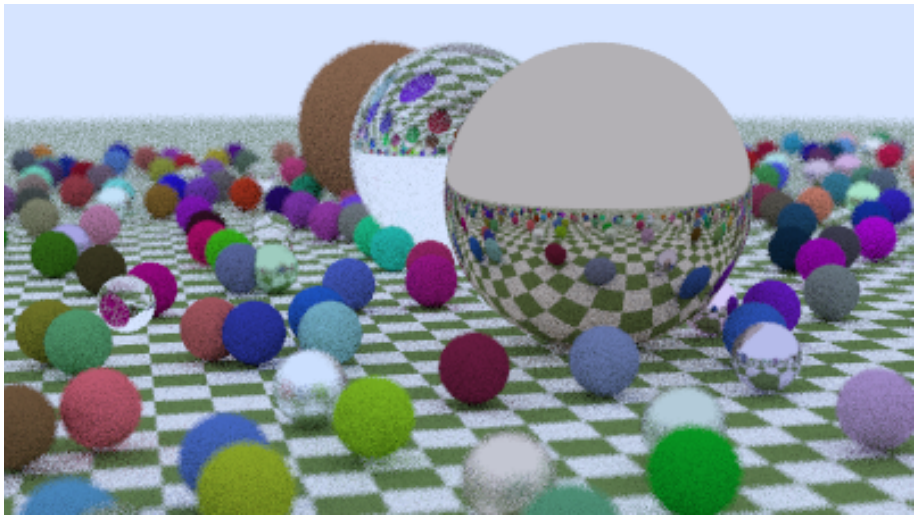
jedna linija, a u `final_scene()` su oba `bvh_node` objekta (kutije poda i oblak sfera) zamijenjena običnom listom `hittable_list`. U oba slučaja se tada svi objekti testiraju jedan po jedan. Obje scene smo za ovo poređenje renderovali s 20 uzoraka po pikselu, jer bi finalna scena bez BVH-a s 250 uzoraka trajala oko 15 minuta. Mjerenje smo izveli na računaru s procesorom Apple M5, uz kompajler Apple clang 21 s optimizacijom `-O3`, na jednoj niti. Rezultati su u tabeli 3.

Tabela 3. Vrijeme renderovanja (za `bouncing_spheres` srednja vrijednost dva pokretanja).

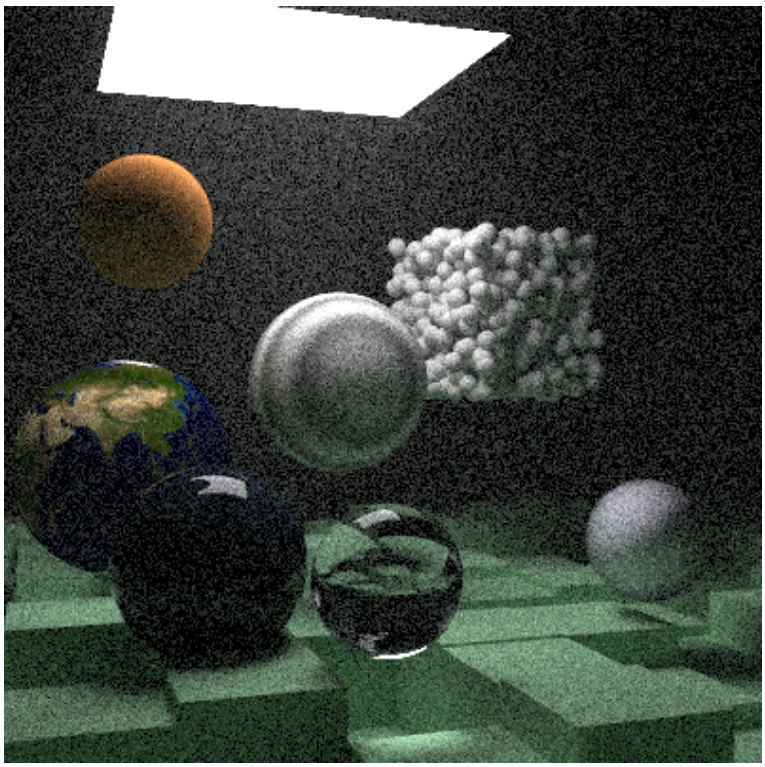
Scena	**Rezolucija**	**Uzorci / dubina**	**Bez BVH**	**Sa BVH**
`bouncing_spheres` (~480 sfera)	400 × 225	20 / 50	2,74 s	0,97 s
`final_scene` (~3400 primitiva)	400 × 400	20 / 4	73,2 s	1,93 s

Na sceni `bouncing_spheres` BVH skraćuje vrijeme približno 2,8 puta, a na sceni `final_scene` približno 38 puta. Razlika je posljedica broja i rasporeda objekata. U finalnoj sceni bez BVH-a svaka zraka testira 400 kutija od po šest četverouglova (2400 testova) i 1000 malih sfera, dok s BVH-om obilazi samo mali broj čvorova duž svoje putanje. U sceni `bouncing_spheres` je objekata manje, pa je i ušteda manja. Bez BVH-a svaka zraka se testira protiv svih objekata, ali dio vremena otpada na posao koji ne zavisi od broja objekata: generisanje zraka, rasijanje na materijalima i generator slučajnih brojeva. Osim toga, stablo se obilazi preko virtuelnih poziva i pokazivača `shared_ptr`, što ima svoj trošak. Ovi rezultati potvrđuju da ubrzanje raste s brojem objekata; tu zavisnost ćemo sistematski izmjeriti u narednoj fazi. Za poređenje, brza testna verzija finalne scene (250 uzoraka po pikselu) s BVH-om traje 23,8 s. Renderovane slike su prikazane na slikama 1 i 2.

Nakon uvođenja prekidača mjerenje smo ponovili bez izmjene koda, sa zadanim parametrima scene 1 (100 uzoraka po pikselu): bez BVH-a renderovanje traje 13,5 s, a s BVH-om 4,7 s, što je ubrzanje od približno 2,9 puta i odgovara rezultatu iz tabele 3. Slike dobijene s BVH-om i bez njega su identične do bajta. To je i očekivano: generator slučajnih brojeva nema postavljen početni broj (seed), pa svako pokretanje gradi istu scenu, a izgradnja i obilazak BVH-a ne troše slučajne brojeve. Time je potvrđeno da BVH samo ubrzava pronalaženje najbližeg presjeka, a ne mijenja rezultat renderovanja.



Slika 1. Scena bouncing_spheres renderovana s BVH-om, 20 uzoraka po pikselu.



Slika 2. Scena final_scene u brzoj testnoj verziji (scena 0 u meniju): 250 uzoraka po pikselu, dubina 4.

8. Naredni koraci

Na osnovu pročišćenog koda u sljedećoj fazi planiramo:

- korištenje odvojenog mjerenja izgradnje i renderovanja, kako je predloženo u Checkpointu 1 [4], uz skriptu koja pomoću argumenata komandne linije pokreće sve scene s BVH-om i bez njega i upisuje vremena u CSV datoteku;
- provjeru builda i automatskog otvaranja slike na Windowsu (MSVC), jer su izmjene iz sekcije 6 testirane samo na macOS-u;
- scene s većim i podesivim brojem objekata, radi mjerenja kako ubrzanje raste s veličinom scene;
- paralelizaciju renderovanja po redovima slike pomoću OpenMP-a; pri tome generator slučajnih brojeva, koji sada koristi `std::rand()`, treba zamijeniti generatorom po niti, jer `std::rand()` nije bezbjedan za istovremeno korištenje iz više niti;
- poređenje trenutne podjele po medijani s podjelom zasnovanom na SAH-u [3, 5].

9. Literatura

[1] Shirley, P., Black, T. D., Hollasch, S. Ray Tracing: The Next Week. raytracing.github.io/books/RayTracingTheNextWeek.html.

[2] Shirley, P., Black, T. D., Hollasch, S. Ray Tracing in One Weekend (serijal i kod). github.com/RayTracing/raytracing.github.io.

[3] Wald, I., Woop, S., Benthin, C., Johnson, G. S., Ernst, M. (2014). Embree: A Kernel Framework for Efficient CPU Ray Tracing. ACM Transactions on Graphics. embree.org/papers/2014-Siggraph-Embree.pdf.

[4] Waldemarson, G., Doggett, M. (2025). Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP. GRAPP/VISIGRAPP. doi:10.5220/0013317100003912.

[5] Wang, C. i sar. (2022). An Improved Ray Tracing Acceleration Algorithm Based on Bounding Volume Hierarchies. arXiv:2208.10008.

[6] Barrett, S. stb: single-file public domain libraries for C/C++ (stb_image, stb_image_write). github.com/nothings/stb.